

Relative path: run_pipeline.py

Filename: run_pipeline.py Extension: .py

```
"""
OASIS v2 – run_pipeline.py
Runs Layers 0-3 end to end, sweeps the flagging threshold for best F1, and
keeps classification accuracy isolated from Layer 1's false positives.
"""

from __future__ import annotations

import json
import time
from pathlib import Path

from classification.classifier import AnomalyClassifier
from detection.model import DetectionLayer
from eval import metrics as ev
from scoring.risk_engine import RiskEngine

DATA_DIR = Path(__file__).resolve().parent / "data"
RESULTS_DIR = Path(__file__).resolve().parent / "results"
RESULTS_DIR.mkdir(exist_ok=True)

THRESHOLD_SWEEP = [0.30, 0.35, 0.40, 0.45, 0.50, 0.55]

def load_data():
    sessions = [json.loads(l) for l in open(DATA_DIR / "sessions.jsonl")]
    sessions.sort(key=lambda s: s["start_time"])
    ground_truth = json.load(open(DATA_DIR / "ground_truth.json"))
    return sessions, ground_truth

def compute_device_recurrence(sessions):
    """Batch-only: counts how many times each (entity_id, device_fingerprint)
    pair appears across the full dataset. A spoofed device_fp appears once;
    a legitimately adopted new device recurs across many sessions. Not
    available in a true streaming deployment without a probation window –
    see Section 10."""
    counts = {}
    for s in sessions:
        key = (s["entity_id"], s["device_fingerprint"])
        counts[key] = counts.get(key, 0) + 1
    return counts

def main():
    t0 = time.time()
    print("=" * 70)
    print("OASIS v2 pipeline run – Layers 0-3")
    print("=" * 70)

    sessions, ground_truth = load_data()
    sessions_by_id = {s["session_id"]: s for s in sessions}
    device_recurrence = compute_device_recurrence(sessions)
    print(f"Loaded {len(sessions)} sessions.")

    normal_sessions = [s for s in sessions if ground_truth[s["session_id"]]["label"] == "normal"]
    print(f"\n[Layer 0+1] Training on {len(normal_sessions)} normal-only sessions...")
    detector = DetectionLayer(cold_start_min_sessions=5, embed_dim=8, hidden_dim=16, max_len=15)
    detector.fit(normal_sessions, epochs=4, lr=0.02)

    print("\n[Layer 0+1] Scoring ALL sessions in timestamp order...")
    anomaly_events = detector.score_all(sessions)
    with open(RESULTS_DIR / "anomaly_events.json", "w") as f:
        json.dump(anomaly_events, f, indent=2)

    print("\n[Eval] Threshold sweep...")
    sweep_results = []
    for t in THRESHOLD_SWEEP:
        prf_t = ev.precision_recall_f1(anomaly_events, ground_truth, threshold=t)
        sweep_results.append(prf_t)
        print(f"    threshold={t:.2f}  P={prf_t['precision']:.3f}  R={prf_t['recall']:.3f}  F1={prf_t['f1']:.3f}")
    best = max(sweep_results, key=lambda r: r["f1"])
    FLAG_THRESHOLD = best["threshold"]
```

```

print(f" -> using threshold={FLAG_THRESHOLD} (best F1={best['f1']})")

print("\n[Layer 2] Training classifier on ground-truth attack-labeled sessions only...")
attack_sessions, attack_labels = [], []
for s in sessions:
    gt = ground_truth[s["session_id"]]
    if gt["label"] == "attack":
        attack_sessions.append(s)
        attack_labels.append(gt["anomaly_type"])
known_resources_by_entity = {eid: p.known_resources for eid, p in detector.profiler.profiles.items()}
known_geos_by_entity = {eid: p.known_geos for eid, p in detector.profiler.profiles.items()}
anomaly_by_id = {a["session_id"]: a for a in anomaly_events}
classifier = AnomalyClassifier()
classifier.fit(
    attack_sessions,
    attack_labels,
    known_resources_by_entity,
    known_geos_by_entity,
    anomaly_by_id,
    device_recurrence,
)
print(f" trained on {len(attack_sessions)} attack sessions across {len(classifier.classes_)} classes:
"
      f"{classifier.classes_}")

flagged = [ev_ for ev_ in anomaly_events if ev_["anomaly_score"] >= FLAG_THRESHOLD]
print(f"\n[Layer 2] Classifying {len(flagged)} flagged sessions (of {len(anomaly_events)} total)...")
classified_events = []
for ev_ in flagged:
    session = sessions_by_id[ev_["session_id"]]
    known_res = known_resources_by_entity.get(ev_["entity_id"], set())
    known_geos = known_geos_by_entity.get(ev_["entity_id"], set())
    anomaly_type, confidence, class_probs = classifier.predict(
        session,
        known_res,
        known_geos,
        anomaly_by_id.get(ev_["session_id"]),
        device_recurrence,
    )
    classified_events.append({
        "session_id": ev_["session_id"],
        "anomaly_type": anomaly_type,
        "type_confidence": round(confidence, 4),
        "class_probabilities": class_probs,
    })
with open(RESULTS_DIR / "classified_events.json", "w") as f:
    json.dump(classified_events, f, indent=2)

print("\n[Layer 3] Computing risk assessments...")
engine = RiskEngine()
risk_assessments = []
anomaly_by_id = {a["session_id"]: a for a in anomaly_events}
for c in classified_events:
    session = sessions_by_id[c["session_id"]]
    ev_ = anomaly_by_id[c["session_id"]]
    risk_assessments.append(engine.score(ev_, c, session["resources_touched"]))
risk_assessments.sort(key=lambda r: -r["impact_score"])
with open(RESULTS_DIR / "risk_assessments.json", "w") as f:
    json.dump(risk_assessments, f, indent=2)

print("\n[Eval] Computing metrics...")
prf = ev.precision_recall_f1(anomaly_events, ground_truth, threshold=FLAG_THRESHOLD)
p_at_1pct = ev.precision_at_k_percent(anomaly_events, ground_truth, k_percent=1.0)

tp_classified = ev.true_positive_classified_events(classified_events, ground_truth)
confusion = ev.classification_confusion_matrix(tp_classified, ground_truth)
fp_misrouting = ev.false_positive_misrouting(classified_events, ground_truth)

cold_start = ev.cold_start_eval(anomaly_events, sessions_by_id)
drift = ev.concept_drift_eval(anomaly_events, ground_truth)

all_metrics = {
    "threshold_sweep": sweep_results,
    "chosen_threshold": FLAG_THRESHOLD,
    "overall_precision_recall_f1": prf,
    "precision_at_top_1_percent": p_at_1pct,

```

```

    "per_class_classification_true_positives_only": confusion["per_class"],
    "false_positive_misrouting": fp_misrouting,
    "cold_start": cold_start,
    "concept_drift_insider_drift": drift,
    "runtime_sec": round(time.time() - t0, 2),
    "n_sessions_total": len(sessions),
    "n_sessions_flagged": len(flagged),
}
metrics_path = RESULTS_DIR / "metrics.json"
if metrics_path.exists():
    existing_metrics = json.load(open(metrics_path))
    if "grounded_narratives" in existing_metrics:
        all_metrics["grounded_narratives"] = existing_metrics["grounded_narratives"]
with open(metrics_path, "w") as f:
    json.dump(all_metrics, f, indent=2)

drift_entity_ids = {s["entity_id"] for s in sessions
                    if ground_truth[s["session_id"]]["anomaly_type"] == "insider_drift"}
if drift_entity_ids:
    eid = sorted(drift_entity_ids)[0]
    with open(RESULTS_DIR / "worked_example_drift_profile.json", "w") as f:
        json.dump({"entity_id": eid,
                  "final_profile_snapshot": detector.profiler.entity_snapshot(eid),
                  "concept_drift_scores": drift}, f, indent=2)

print("\n" + "=" * 70)
print("SUMMARY")
print("=" * 70)
print(f"Detection @ threshold {FLAG_THRESHOLD}: precision={prf['precision']} "
      f"recall={prf['recall']} f1={prf['f1']} (tp={prf['tp']} fp={prf['fp']} fn={prf['fn']})")
print(f"Precision@top-1%: {p_at_1pct['precision_at_k']} "
      f"({p_at_1pct['hits']}/{p_at_1pct['k_sessions']} of top {p_at_1pct['k_percent']}% were real attacks)")
print("Per-class classification, TRUE POSITIVES ONLY (precision/recall/support):")
for cls, m in confusion["per_class"].items():
    print(f"    {cls:22s} P={m['precision']:.2f} R={m['recall']:.2f} n={m['support']}")
print(f"False-positive misrouting: {fp_misrouting['n_false_positives_classified']} FPs, "
      f"routed to: {fp_misrouting['misrouted_to']}")
print("Cold-start buckets (mean anomaly_score by # prior sessions seen):")
for bucket, m in cold_start.items():
    print(f"    {bucket:15s} n={m['n']:.4d} mean_score={m['mean_anomaly_score']}")
print(f"Concept drift (insider_drift entity): mean_early={drift.get('mean_early')} "
      f"-> mean_late={drift.get('mean_late')} adapted={drift.get('adapted')}")
print(f"\nRuntime: {all_metrics['runtime_sec']}s")
print(f"Results written to {RESULTS_DIR}")

if __name__ == "__main__":
    main()

```